

Secure by default.

3.5× cheaper, 3.8× faster.

Three agents, one machine, identical prompts: the bloated original is 3.5× dearer and 3.8× slower per task. Eight AgentDojo tasks (MIT, real data) — one run of our script, reproducible end to end.

The problem

Autonomous agents are being handed real state — files, network, processes — while nobody can say what they will do next. Securing that means reviewing everything: every token the agent emits, every byte a tool hands back. That review is the security boundary, and a bloated agent makes it unaffordable — 1,900 output tokens and 80K of re-sent context for one small task. Our lab builds secure stateful execution for agents: capability-based permissions, bounded delegation, provenance. None of it is practical on a bloated runtime. Make the agent small enough to audit, and you can afford to audit everything.

What we changed

Economy in the prompt

Terse, predictable output in the default prompt. Less text per step, less for a reviewer to read.

Capped tool output

Bash capped at 64 KiB, job ring at 128 KiB. Hard bound on what tools can inject into context.

Lean networking

Idle connections reused. Less surface area, cleaner logs.

How we measured it

Fresh workspace per run, deterministic verifiers, identical prompts, DeepSeek v4 flash on one machine, real AgentDojo banking and travel data (MIT). Nothing hand-tuned per task.

Cache-aligned context

Byte-exact prefixes, stable blocks. Reproducible runs, reproducible audits. 42K cache hits per task.

Every run measured

Metrics file per session: tokens, cache, cost, steps. Provenance starts with measurement.

One script proves it

runner.py replays the benchmark; charts regenerate from raw results JSON.

Agent	Tasks	Time / task	Cost / task	Tokens
Our agent (highly optimized)	8/8	5.4 s	0.21¢	271
Original agent (bloated baseline)	8/8	20.2 s	0.73¢	1,935
Hermes agent (third party)	8/8	18.8 s	0.06¢*	679

* Hermes' own usage estimate, cache-priced; its real prompt volume hides in 89K cache reads per task. Different pricing path, so we compare against the original.

Every action traced, every tool bounded. 0.21¢ a task.

3.5× cheaper and 3.8× faster than the bloated original. Output shrank 7× — less to review, less for anything untrusted to reach.